

CSCI 210: Computer Architecture

Lecture 11: Procedures

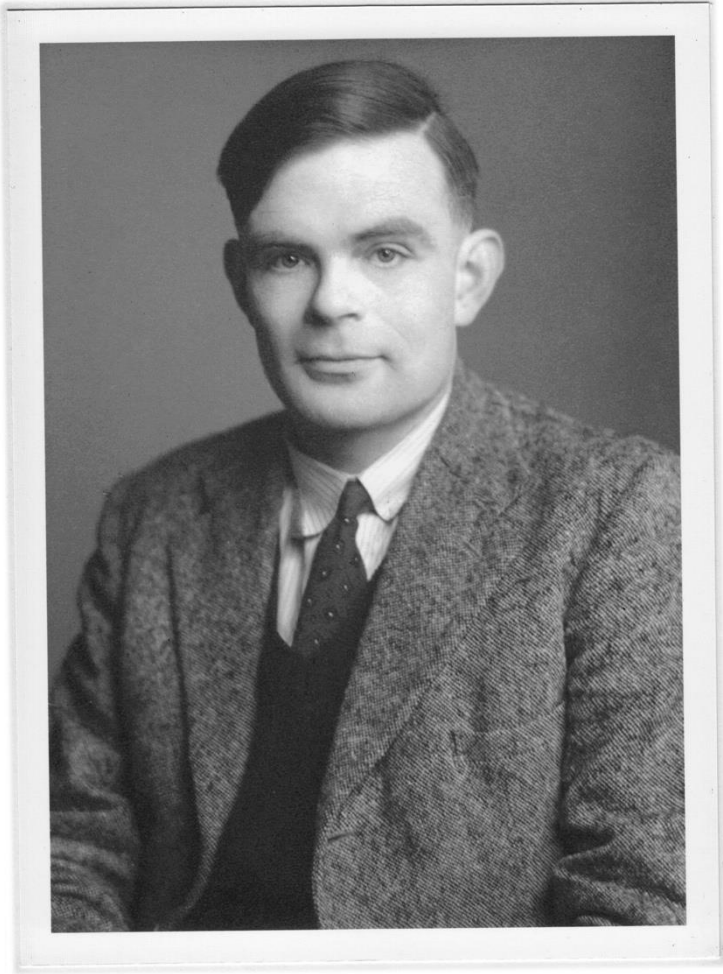
Stephen Checkoway

Slides from Cynthia Taylor

Announcements

- Problem Set 3 Due Friday
- Lab 2 due Sunday

CS History: The Subroutine



- A group of instructions we can re-run as a unit
- Conceived of by Alan Turing in 1945, independently implemented by Kay McNulty and others on the ENIAC in 1947, formally developed by Maurice Wilkes, David Wheeler, and Stanley Gill in 1952.
- In early computers, loaded as strips of paper tape or collections of punch cards that would be reinserted into the machine
- Later developed as macros, pieces of code the assembler would copy into multiple places during assembly

Subroutines/functions: A high-level view

- Code in programs is organized in functions
- Functions take arguments
- Functions can call functions, including themselves
- Functions have local variables that are not shared with other functions, including other invocations of the same function (i.e., recursive calls to a function have different local variables)
- Functions return to the function that called them

Implications

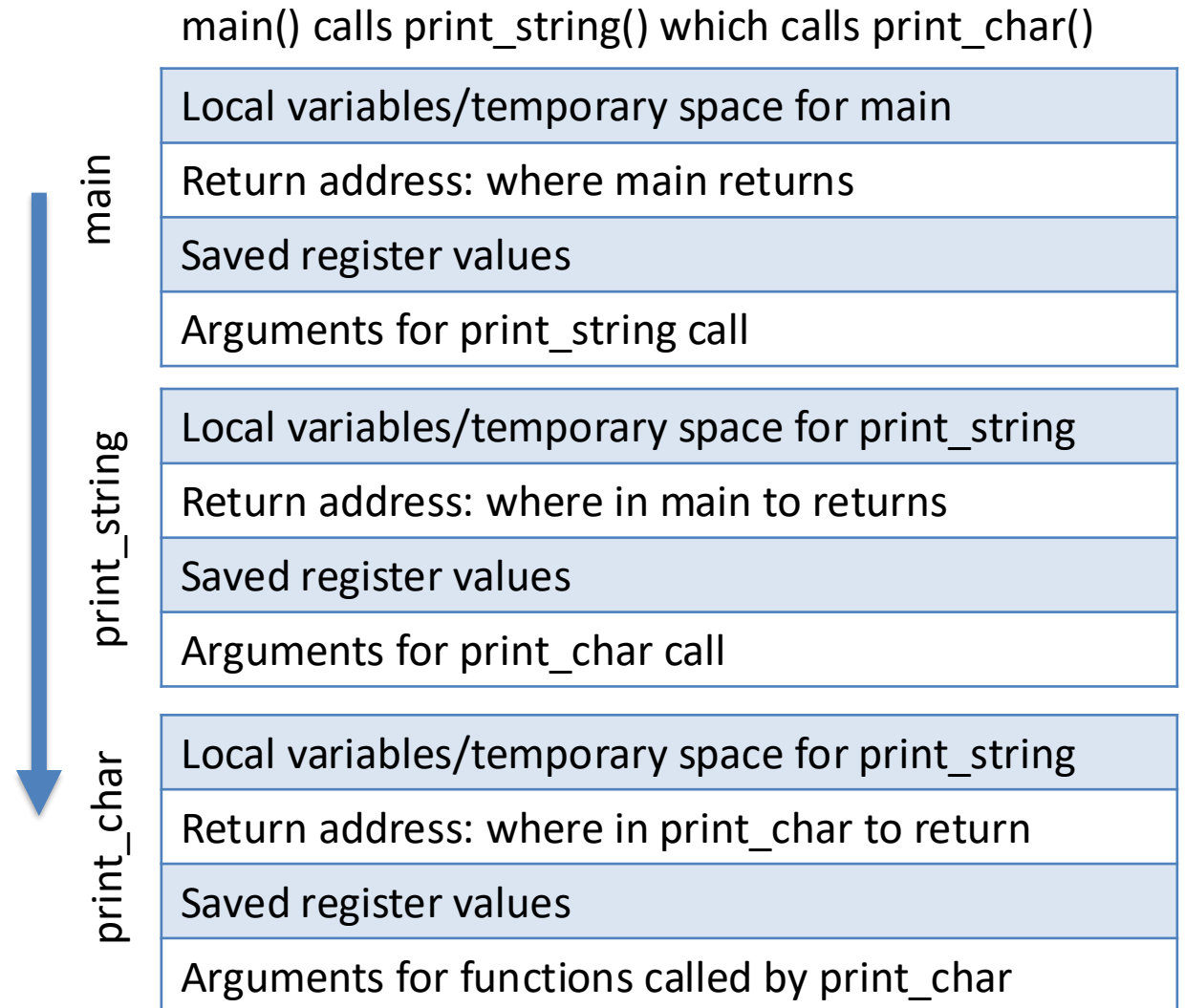
- Functions take arguments: Need a way to access arguments
- Functions can call functions: Need a way to pass arguments
- Functions have local variables: Need per-function-call memory to hold the variables
- Functions return: Need to know what the return address is

Activation Records

- A per-function-call data structure that holds
 - Local variables/temporary storage space
 - Return address
 - Saved register values
 - Arguments for the next function call

Stack of activation records

- Each time a function is called, a new activation record is pushed onto a stack
- Each time a function returns, the activation record is popped off the stack



From theory to practice

- Activation record is the name we give to the data structure
- A **stack frame** is how an activation record is realized in software
- In RISC-V, the stack frame for a called function is created by the called function itself by decrementing the stack pointer register `sp` as the first thing a function does
- The return address and any register values that need to be saved are stored on the stack using `sd reg, offset(sp)`

Recall from Last Class

- Fetch/Decode/Execute cycle
 - $IR = \text{Memory}[PC]$
 - $PC = PC + 4$
- Branch instructions change PC value conditionally
 - `beq, bne, blt, bge`
- Jump instructions always change PC value
 - `j, jal, jalr`

Jump and Link (register)

- `jal rd, label`
 - Address of following instruction put in `rd`
 - Jumps to target address given by label
- `jalr rd, offset(rs)`
 - Address of following instruction put in `rd`
 - Jumps to target address given in `rs + offset`

What is the most common use of a `jal` instruction with the destination register something other than `x0` (the `zero` register) and why?

	Most common use	Best answer
A	Procedure call	<code>jal rd, label</code> stores the address of the next instruction in the current function in register <code>rd</code> so the function at address <code>label</code> knows where to return to
B	Procedure call	<code>jal rd, label</code> enables a long jump to the function at address <code>label</code> and most procedures are a fairly long distance away
C	If/else	<code>jal rd, label</code> lets you go to the “then branch” at address <code>label</code> while storing <code>pc+4</code> in <code>rd</code> for the “else branch”
D	If/else	<code>jal rd, label</code> enables a long branch and most if statements are a fairly long distance away
E	None of the above	

Procedure Call Instructions

`ra` is the Return Address register `x1`,
not a place holder like `rs1` or `rd`



- Procedure call: jump and link

```
jal ra, ProcedureLabel
```

- Address of following instruction put in `ra` (the hardware allows any register to be used, `ra` is the right one for making function calls)
- Jumps to target address

- Procedure return: jump and link register

```
jalr zero, 0(ra)
```

- Copies `ra+0` to program counter
- By using `zero` as `rd`, the address of the following instruction is not saved in any register

Procedure call pseudoinstructions

Procedure call jumps to `label` and sets the return address register

`call label` $\Rightarrow$ `jal ra, label`

Procedure return jumps to the return address and sets nothing

`ret` $\Rightarrow$ `jalr zero, 0(ra)`

There's also jump register

`jr rs` $\Rightarrow$ `jalr zero, 0(rs)`

Procedure Calling

1. Place arguments in registers: `a0, a1, ... , a7`
2. Transfer control to procedure: `jal ra, label`
(or `call label`)
3. Allocate stack frame for procedure (when necessary)
4. Perform procedure's operations
5. Place result in register for caller: `a0`
6. Deallocate the stack frame (when allocated)
7. Return to place of call: `jalr zero, 0(ra)`
(or `ret` or `jr ra`)

Making a procedure call

Explicit jal/jalr instructions

```
addten:
    addi    a0, a0, 10
    jalr    zero, 0(ra)

main:
    # ...
    add     a0, zero, s2
    jal     ra, addten
    # Now a0 holds s2+10
```

call/ret pseudoinstructions

```
addten:
    addi    a0, a0, 10
    ret

main:
    # ...
    mv      a0, s2
    call    addten
    # Now a0 holds s2+10
```

What, if anything, is wrong with this code

```
mv      a0, t2
mv      a1, t3
call    add
mv      t4, a0
sub     t4, t4, t2

#add(a0, a1)
add:
        add     t2, a0, a1
        mv      a0, t2
        ret
```

A. Not adding correctly

B. t2 is overwritten in add

C. add is not saving the return address before the procedure returns

D. There is nothing wrong with this code

E. There's something else wrong with the code

Register values across function calls

- “Preserved” registers
 - You can trust them to persist past function calls
 - Functions must not change them or they must **restore them if they do**
- Not “Preserved” registers
 - Contents can be changed when you call a function
 - If you need the value, you need to put it somewhere else

RISC-V Register Convention

Programmer's
responsibility



Name	ABI Mnemonic	Meaning	Preserved across calls?
x0	zero	Zero	— (Immutable)
x1	ra	Return address	No
x2	sp	Stack pointer	Yes
x3	gp	Global pointer	— (Unallocatable)
x4	tp	Thread pointer	— (Unallocatable)
x5 – x7	t0 – t2	Temporary registers	No
x8 – x9	s0 – s1	Callee-saved registers	Yes
x10 – x17	a0 – a7	Argument registers	No
x18 – x27	s2 – s11	Callee-saved registers	Yes
x28 – x31	t3 – t6	Temporary registers	No

We won't modify `gp` or `tp`; signal handlers (remember 241?) rely on their values

“Spill” and “Fill”

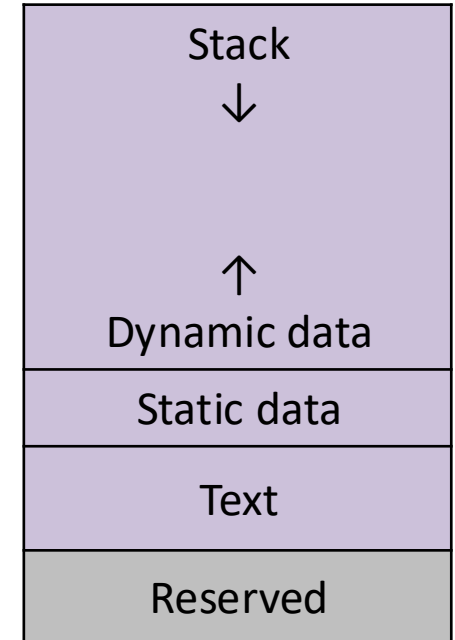
- Spill register **to** memory
 - Whenever you have too many variables to keep in registers
 - Whenever you call a method and need values in non-preserved registers
 - Whenever you want to use a preserved register and need to keep a copy
- Fill registers **from** memory
 - To restore previously spilled registers

Memory Layout

sp → 3FF FFFFFFFF

- Text: program code
- Static data: global variables
 - e.g., static variables in C, constant arrays and strings
- Dynamic data: heap
 - E.g., malloc() in C, new in Java, Box::new() in Rust
- Stack: “automatic” storage for procedures

pc → 10000
0



Before and after a function

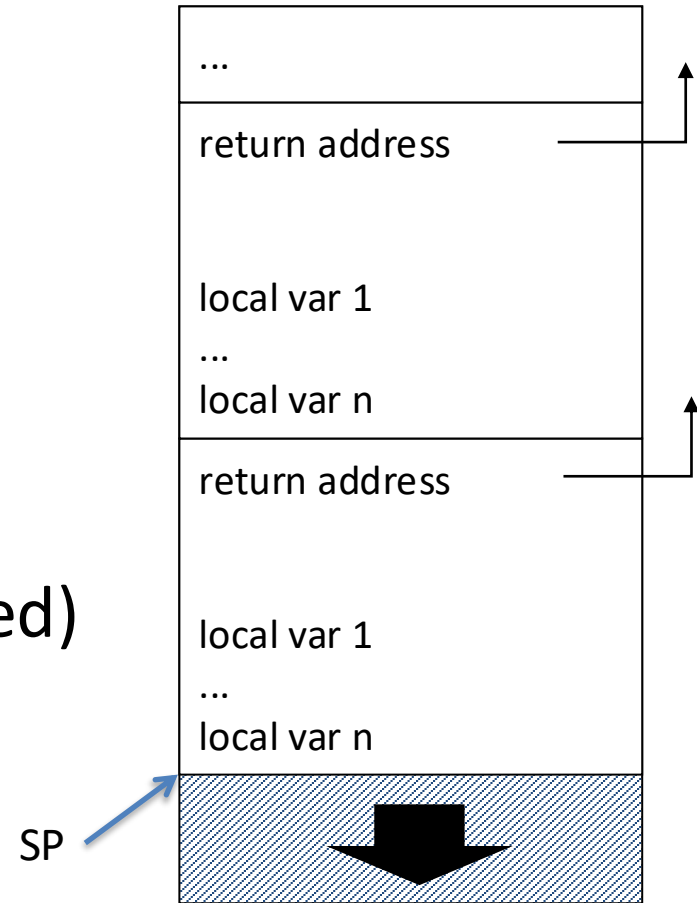
```
sd      t0, 24(sp)
jal     ra, some_function
ld      t0, 24(sp)
```

Which register is being spilled and filled?

- A. ra
- B. t0
- C. sp
- D. gp
- E. tp

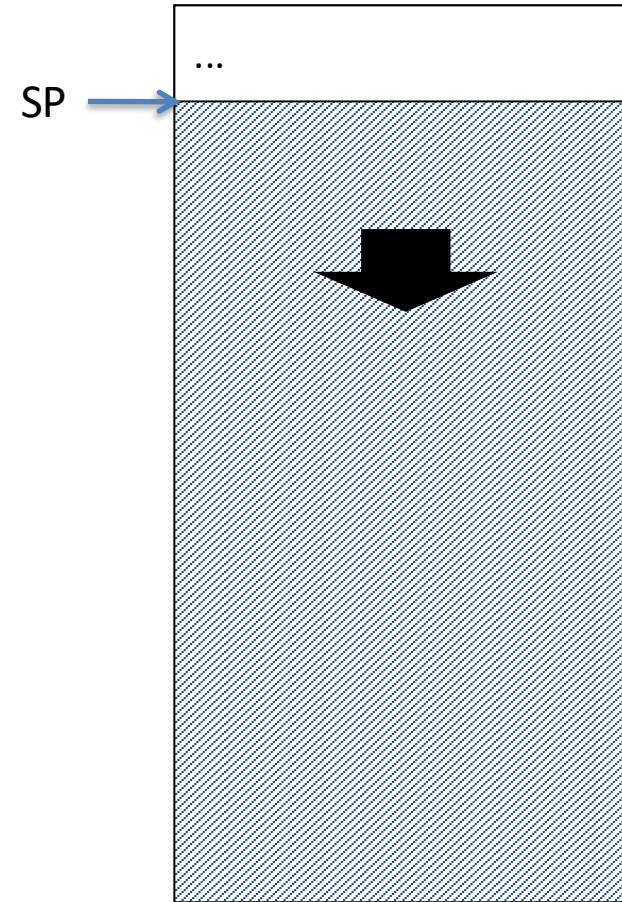
Stack

- Stack of stack frames
 - One per pending procedure
- Each stack frame stores
 - Where to return to
 - Local variables
 - Arguments for called functions (if needed)
- Stack pointer points to last record

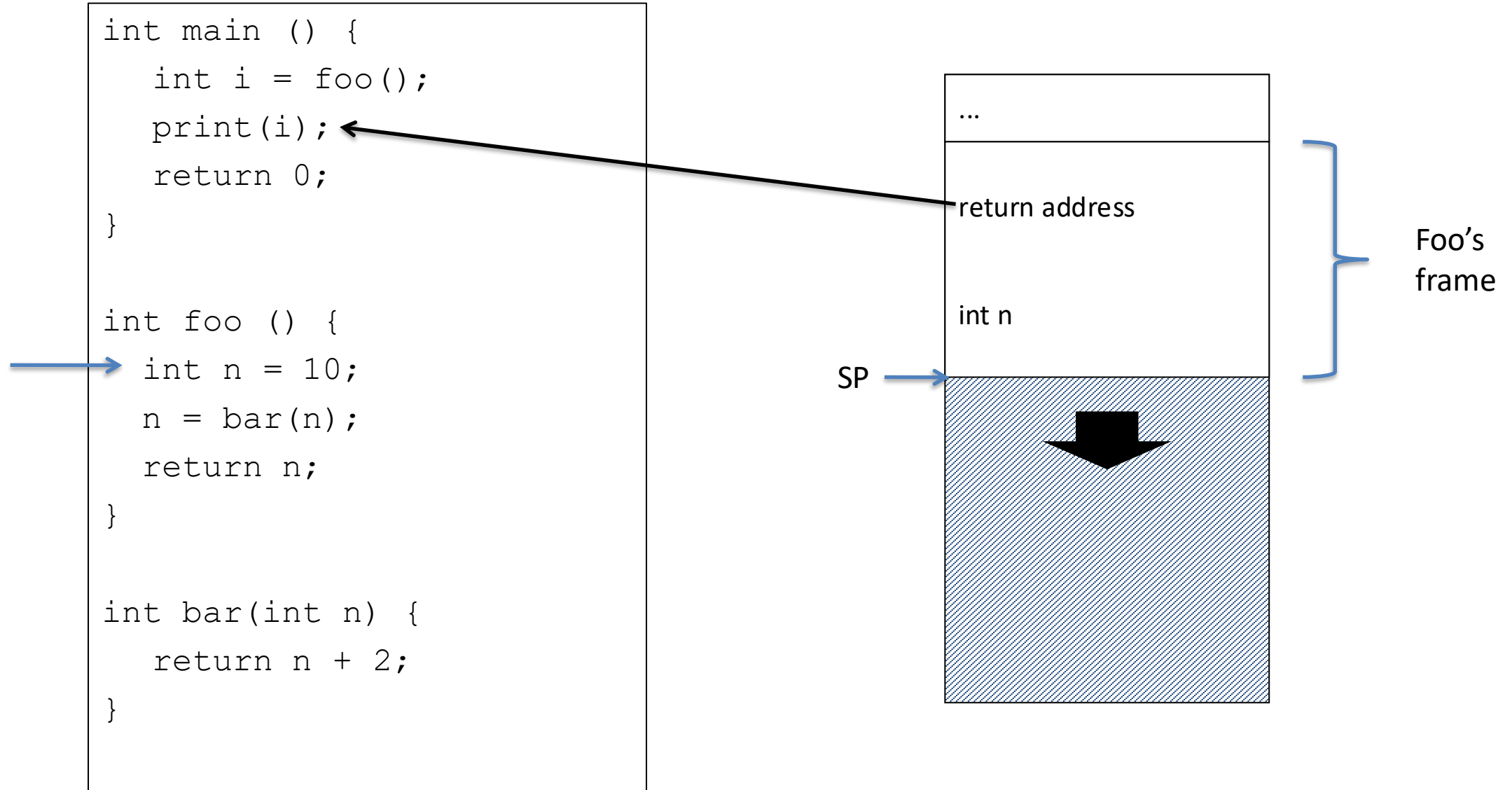


Process Stack

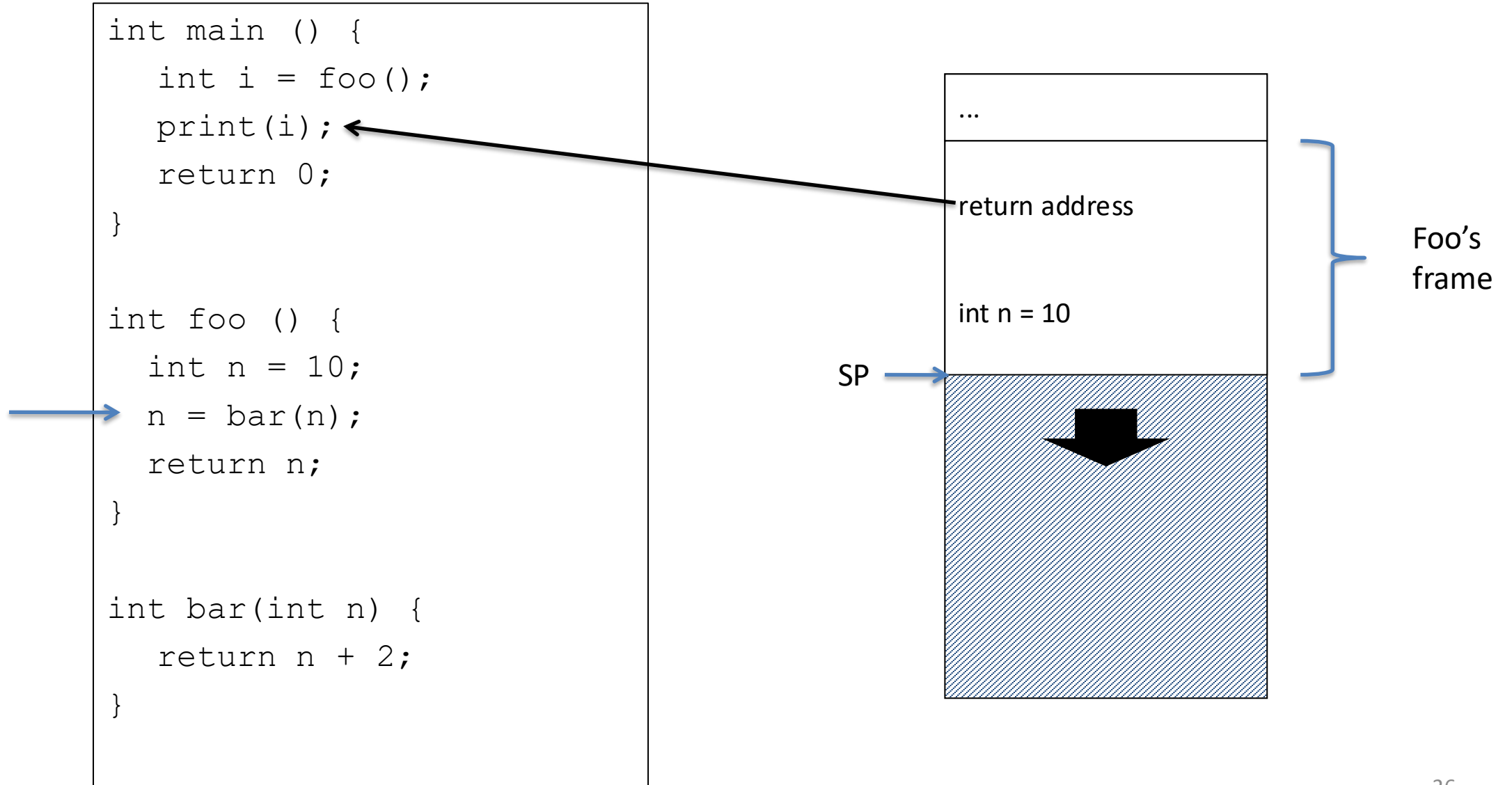
```
int main () {  
    → int i = foo();  
    print(i);  
    return 0;  
}  
  
int foo () {  
    int n = 10;  
    n = bar(n);  
    return n;  
}  
  
int bar(int n) {  
    return n + 2;  
}
```



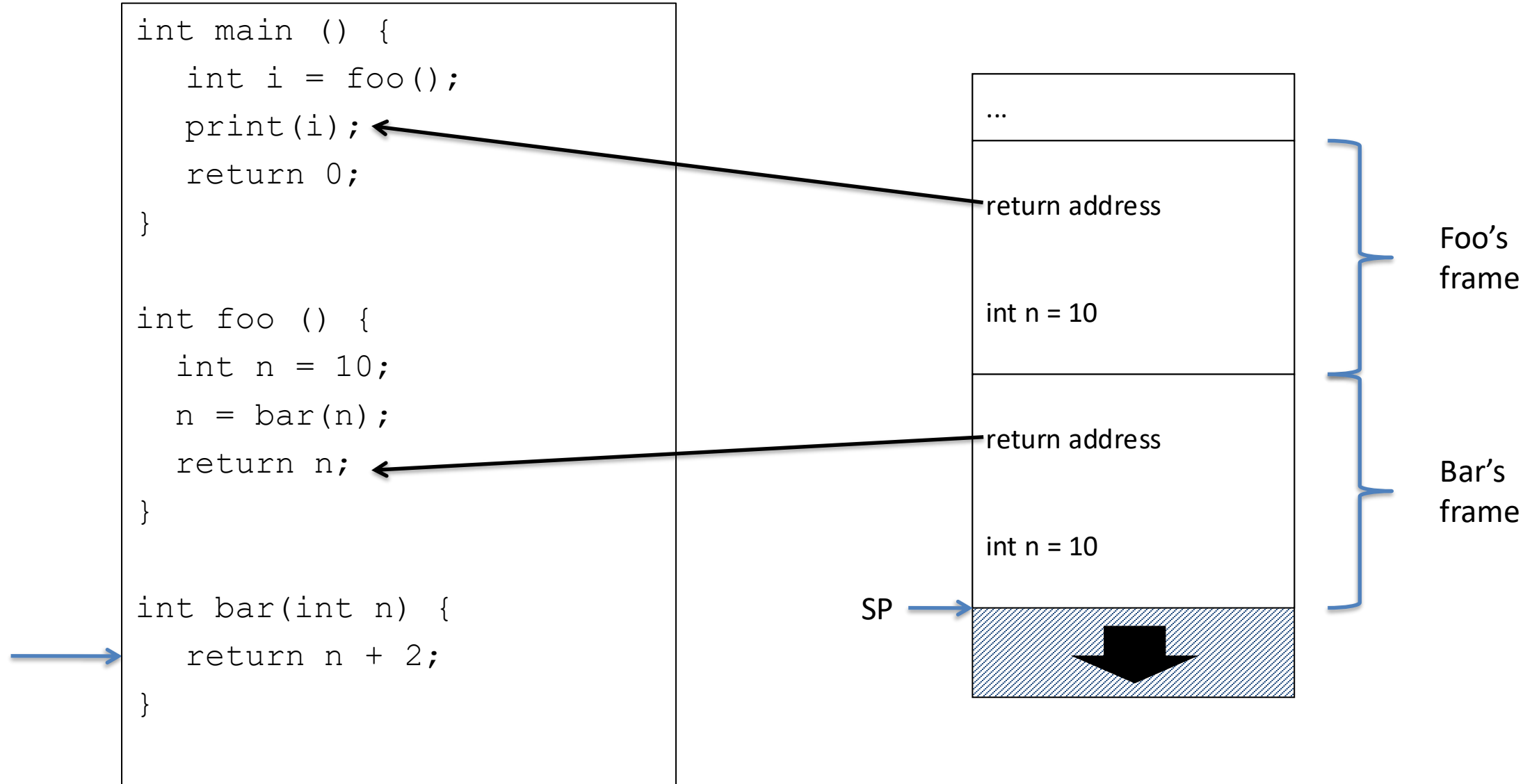
Process Stack



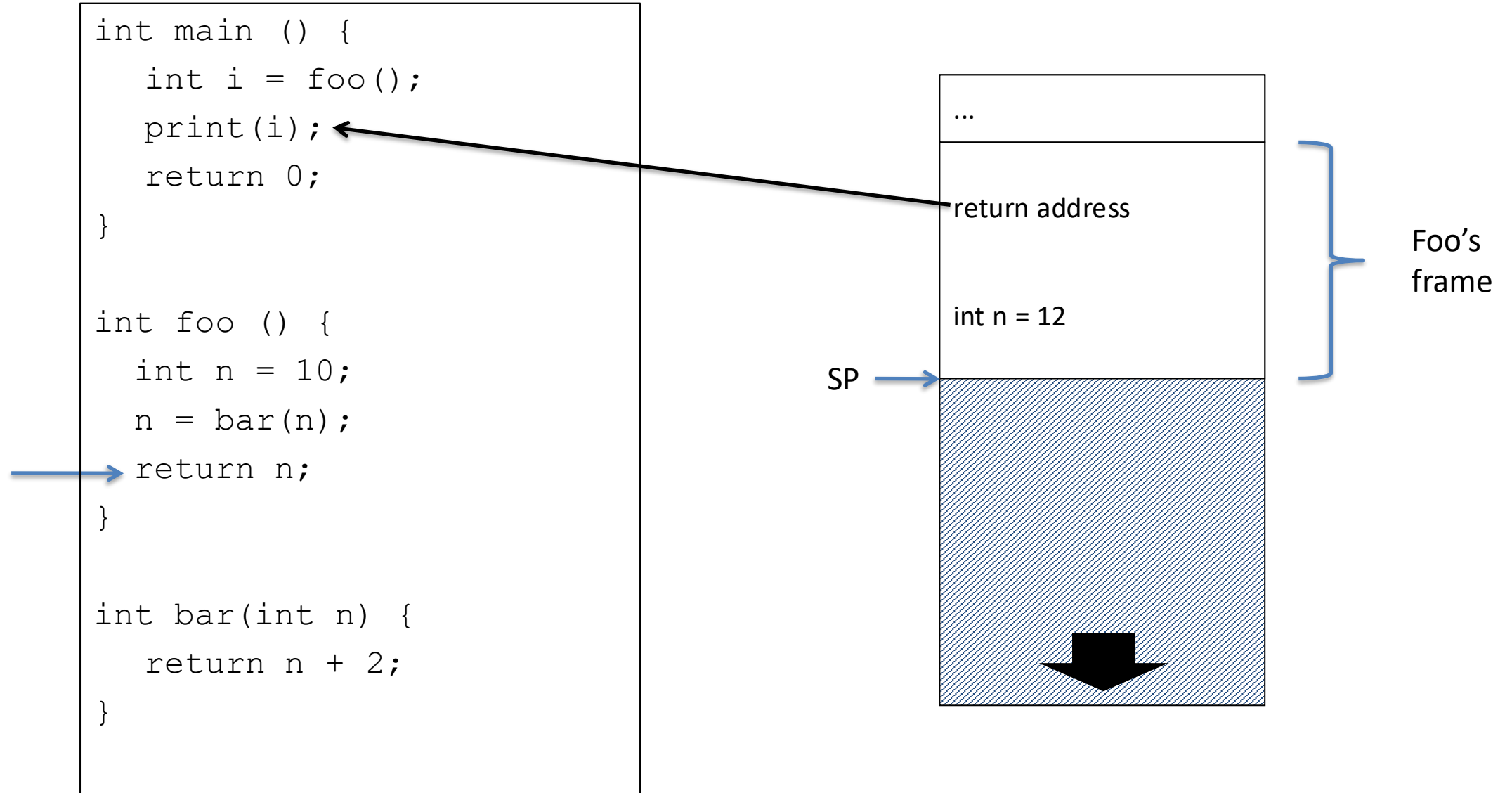
Process Stack



Process Stack

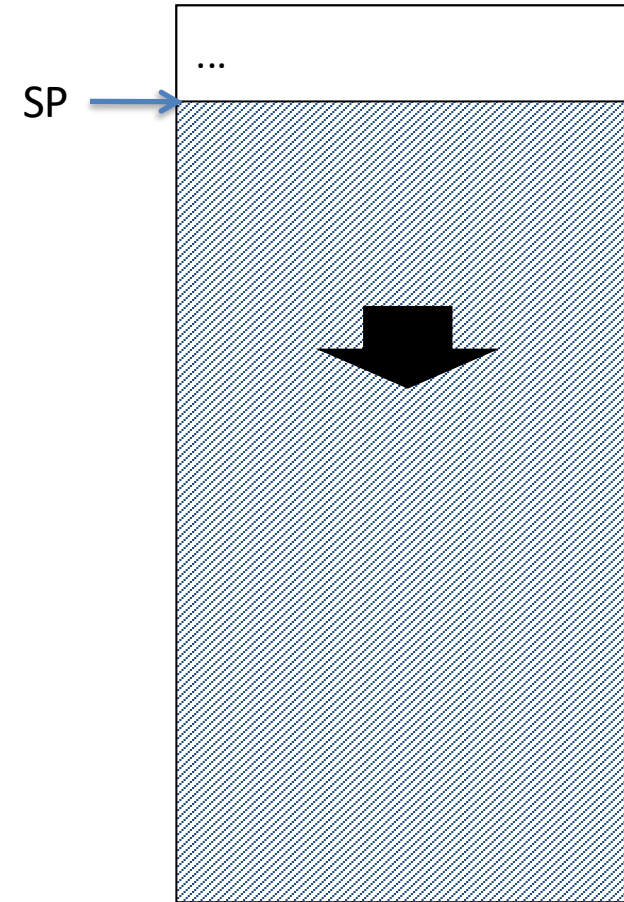


Process Stack



Process Stack

```
int main () {  
    int i = foo();  
    print(i);  
    return 0;  
}  
  
int foo () {  
    int n = 10;  
    n = bar(n);  
    return n;  
}  
  
int bar(int n) {  
    return n + 2;  
}
```



To add a variable to the stack in RISC-V

- Change the stack pointer sp to create room on the stack for the variable
- Use sd to store the variable on the stack

Stack

If you wish to **push** an integer variable to the top of the stack, which of the following is true:

A. You should decrement the stack pointer (sp) by 1

B. You should decrement sp by 8

C. You should increment sp by 1

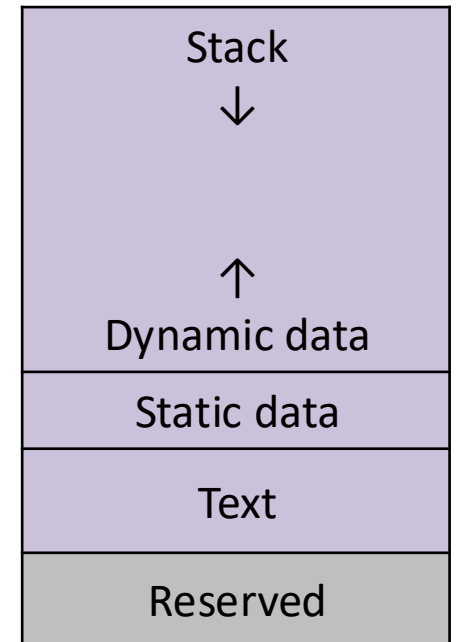
D. You should increment sp by 8

E. None of the above

$sp \rightarrow 3FF\ FFFFFFFF$

$pc \rightarrow 10000$

0



Manipulating the Stack

- To store the contents of s0 to the stack

- `addi sp, sp, -8`
`sd s0, 0(sp)`

- To get the value back from the stack

- `ld s0, 0(sp)`

- To “erase” the value from the stack

- `addi sp, sp, 8`

- [Does this actually erase anything from memory?]

Think-Pair-Share: Why do we spill and fill the return address when we call a function from inside another function?

```
func1:
```

```
    # ...  
    addi    sp, sp, -8  
    sd      ra, 0(sp)  
    jal     ra, func2  
    ld      ra, 0(sp)  
    addi    sp, sp, 8  
    # ...  
    jalr    zero, 0(ra)
```

A better approach

- In the function “prologue,” reserve space on the stack for all of the variables and saved registers you’ll need
- Use `sd/ld` to spill and fill as needed to the space reserved in the prologue
- In the function “epilogue,” restore any saved registers you need and update the stack pointer

Complete example

foo:

```
addi    sp, sp, -32      # Allocate space for stack frame
sd      ra, 24(sp)       # Stores (spills) ra, return address
sd      s0, 16(sp)       # Stores (spills) s0, callee-saved reg
...
li      s0, 25           # Set s0 to 25
sd      t3, 8(sp)        # Stores (spills) t3, caller-saved reg
add     a0, t1, t3
jal     ra, myFunction
ld      t3, 8(sp)        # Restores (fills) t3
...
ld      s0, 16(sp)       # Restores (fills) s0, must restore
ld      ra, 24(sp)       # Restores (fills) ra, return address
addi    sp, sp, 32       # Restore the stack pointer
jalr    zero, 0(ra)      # Return
```

Complete example

foo:

```
addi    sp, sp, -32
sd      ra, 24(sp)
sd      s0, 16(sp)
...
li      s0, 25
sd      t3, 8(sp)
add     a0, t1, t3
jal     ra, myFunction
ld      t3, 8(sp)
...
ld      s0, 16(sp)
ld      ra, 24(sp)
addi    sp, sp, 32
jalr    zero, 0(ra)
```

At the start of the function:

Arguments are in `a0, ..., a7` and then on the stack at `0(sp), 8(sp), ...` at the start of the function

Stack frame for foo (32 bytes in size)

<code>sp + 24</code>	Saved return address <code>ra</code>
<code>sp + 16</code>	Saved register <code>s0</code>
<code>sp + 8</code>	Saved register <code>t3</code>
<code>sp + 0</code>	Unused space to preserve 16-byte alignment

Reading

- Next lecture: More stack!
- Reading: Section 2.14
- Problem Set 3 due today
- Lab 2 due Sunday